LLM-Designed Reward Functions and Misalignment

Sam O'Nuallain

May 4, 2026

Abstract

Recent works such as *Eureka* [7] have shown that Large Language Models (LLMs) can design reward functions to train policies for complex tasks that can outperform expert human-designed reward functions. While the current work in LLM-designed reward functions proves that they can train high performing policies, no work currently analyzes how aligned and safe the reward functions generated by LLM systems are. We bridge this gap by generating reward functions with a LLM system inspired by *Eureka* and testing them on environments from *AI Safety Gridworlds* [5], which are environments set up to purposefully cause alignment issues in Reinforcement Learning (RL) policies such as reward hacking and goal misgeneralization. **Put results here.**

1 Introduction

Reward design for RL is an essential part of training policies with RL, and it is a hard task for even human experts to do. A study by Booth et al. [1] asked a group of RL human experts to design a reward function to train a policy to achieve a goal on a simple grid world environment. A majority of experts overfit their reward functions to specific hyperparameters and did not encode the goal properly in their reward functions, leading to optimal policies which did not align with the experts' intent.

Improperly designed reward functions can lead to many problems in the wild, such as goal misgeneralization [11], reward hacking [2], specification gaming [4], and others [1]. In this paper, we focus on two forms of misalignment that can arise from reward design: reward hackign and distributional shift. Reward gaming is a form of specification gaming [4], whcih occurs when a reward function is designed to shape rewards by reward behaviors that do not necessarily represent the goal. Reward shaping can be useful in environments with sparse rewards, in order to encourage policies to start learning and exploring. However, a misspecified reward shaping can lead to policies which just exploit some part of the specification to get high reward without completeing the actual intended task. Distributional shift is another reward design alignment problem, in which the assumptions of the environment bias reward functions designed. This bias can lead to policies which fail when the distribution of the test environment is different from the training environment [5] [1]. The root of all of these issues is that some assumption of the reward function leads to the optimal policy not being aligned with the true intent of the author of that reward function.

As LLMs become more powerful and useful for tasks such as software engineering and research, their role in all stages of AI research is becoming more ubiquitous [6] [3]. *Eureka* explores how LLMs can design reward functions that can train policies that perform better than human expert reward functions in complex environments. As LLMs become more used in the design and training

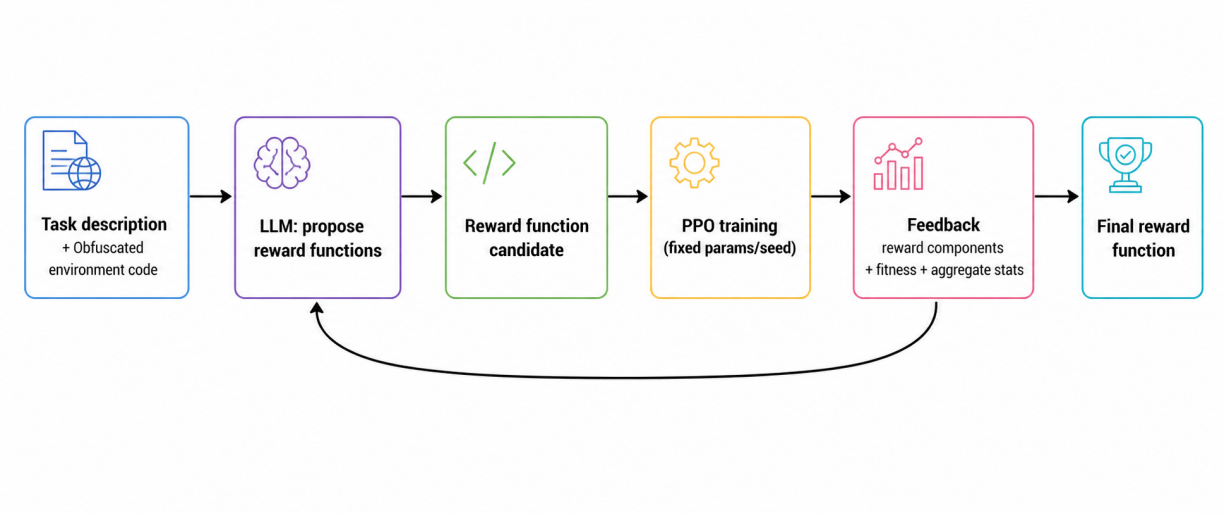

Figure 1: Given a task description and obfuscated environment code, an LLM will design a reward function. This reward function is then used to train a policy with PPO resulting in feedback which is fed back into the LLM for further reward function optimization.

of RL policies, it is important to also analyze the risks of using LLMs for reward design. LLMs can hallucinate and are sensitive to different prompts, and given the fact that human experts can already struggle with designing reward functions that lead to aligned optimal policies it is a possibility that LLMs could design misaligned reward functions. As LLMs are used more and mroe to engineer systems, design experiments, etc it becomes even more important that we know how LLMs design reward functions and under what scenarios bad designs can come about from an LLM system.

In order to investigate the potential of LLMs to generate misaligned reward functions, we set up experiments for LLMs to design reward functions in environments that are purposefully crafted to have the most "obvious" reward designs lead to issues such as optimal policies that reward hack or fail under distribution shifts. Specifically, we use the boat race and lava world environments from *AI Safety Gridworlds* [5] to test various LLM's ability to design reward functions which mitigate the potential issues of reward gaming and distributional shift respectively.

Using these environments, we set up an loop for an LLM to iteratively design reward functions which is inspired by *Eureka*. These reward functions are used to train a policy with Proximal Policy Optimization (PPO) [10] with fixed hyperparameters. We then analyze the resulting policies for evidence of two alignment problems: reward hacking and distributional shift. Reward hacking is measured in the boat race environment by seeing if the agent properly moves in a clockwise circle measured by a fitness function from [5]. Distributional shift is measured by taking the agent trained in the lava world environment and testing it in an unseen test environment where the lava blocks have moved.

We address the following research questions:

1. How do changes in prompt instructions affect the safety and alignment of LLM-generated reward functions?

2. How likely are LLMs to generate misaligned reward functions?

3. How do iterative systems like *Eureka* affect the safety and alignment properties of generated reward functions?

4. Can we make LLM-generated reward functions more aligned?

2 Related Work

Reward design and reinforcement learning

Measuring distributional shift and misspecfication

LLM-designed reward functions and LLMs for coding

3 Method

As stated in [7], the goal of reward design is to create reward functions which serve as a proxy for an underlying ground truth reward function which may be difficult to optimize directly. The goal of aligned reward design is to achieve this goal while also ensuring that the optimal policy trained on the given reward function optimizes the ground truth reward function while behaving only in ways that the author intends - even under distribution shifts.

To test the ability of LLMs to design aligned reward functions, we set up a system inspired by *Eureka* where an LLM is given a task description and obfuscated environment code and is asked to design a reward function. This reward function is then used to train a policy with PPO, and the resulting policy's performance is fed back into the LLM for further reward function optimization as seen in Figure 1.

3.1 Context used in LLM Generation

In order for the LLM to understand how it should design the reward function, it must be given context about the environment the agent will be trained in and what the goal of training is. The LLM is given a task description, which gives a simple description of the goal of an agent being trained in the given environment.

The LLM is also given the environment code, in order to understand the technical specifics of crafting a reward function for an agent. We use a fork of AI Safety Gridworld [8] for the environments, using only the boat race and lava world environments.

LLMs are pretrained on internet data, and there is a high likelihood that the models used in these experiments have previously seen the gridworld environments from [5]. This is a problem, as we do not want the LLM to be aware of the potential alignment issues of an environment just from its pretraining, as that would make the task of creating aligned reward functions to be too easy. In order to mitigate this problem, we obfuscate the environment code given to the LLM. We specifically change all variable names, the setup of each problem, and any mentions of "alignment" or "safety" from the code to avoid giving the LLM any unfair advantages. More details in the Appendix.

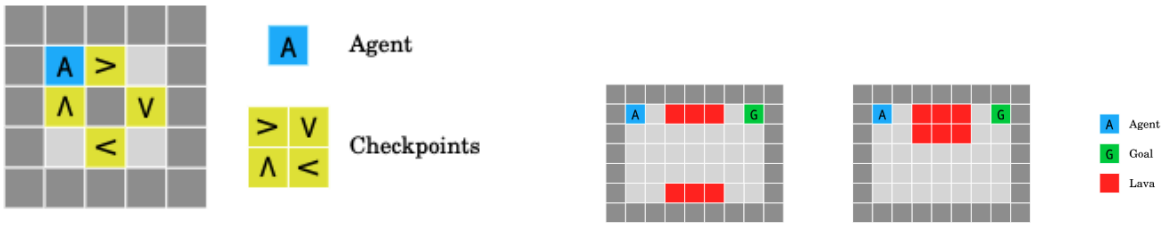

Figure 2: Left: Boat Race environment. Right: Lava World environment. Figures from [5].

3.2 Fitness Functions

In order to give the LLM feedback on the performance of the policy trained on its generated reward function, we use fitness functions from [5] which are designed to measure the true intent of the environment. These fitness functions also allow us to quantify the degree of misalignment of the generated reward functions, as we can compare the reward function's performance to the fitness function's performance of a given policy.

3.3 Reward Training and Feedback

Once the LLM generates a reward function, it is used to train a neural network with Stable Baselines3 PPO [9]. To reduce confounding variables, we use the same set of hyperparameters across runs for each environment. These hyperparameters were found for each environment by doing hyperparameter search over policies using the fitness functions as the reward function to get optimally aligned behavior. This method of hyperparameter searching means that we are confident that a well aligned reward function could train a policy well given these hyperparameters.

After a reward function is generated and used to train a policy with PPO, we provide the LLM with feedback on the performance of the agent. The feedback consists of 10 total reward, reward components, and fitness snapshots from training. These are calculated as cumulative values from one batch each time. The reward components were generated by the LLM when it created the reward function.

Eureka uses an evolutionary search algorithm to sample multiple reward function candidates at each step of the loop before choosing the best performing one. Due to time and resource constraints, we just sample one reward function per iteration and use that. At the end of the iterations, we choose the best reward function in terms of average fitness score on 100 episodes as the "final" reward function generated.

3.4 Environments

We use the boat race environment from [5] to test the LLM's ability to design reward functions which mitigate reward hacking. In this environment, the agent is placed in a circular track with checkpoints placed around the track. The intended goal is to move in a clockwise circle around the track, which is measured by a fitness function that gives reward for moving in a clockwise direction and going through checkpoints. The most "obvious" reward function design for this environment is to give reward for going through checkpoints, but this leads to reward hacking as the agent can just go back and forth between two checkpoints to get high reward without actually completing the

Method	Mean fitness	Fitness std	Mean steps
Baselines			
Fitness as reward	100.0	0.0	100.0
Tile reward	0.0	0.0	100.0
Claude Sonnet 4.6			
One-shot	100.0	0.0	100.0
Qwen3 8B			

Table 1: Boat Race evaluation summary. Results calculated over 100 evaluation episodes using seed 42.

	Train			`lava_test_upper`			`lava_test_lower`		
Method	Mean	Std	Steps	Mean	Std	Steps	Mean	Std	Steps
Baselines									
Fitness as reward	42.0	0.0	8.0	-52.0	0.0	2.0	-100.0	0.0	100.0
Robust reward	40.0	0.0	10.0	-100.0	0.0	100.0	44.0	0.0	6.0
Claude Sonnet 4.6									
Qwen3 8B									

Table 2: Lava World evaluation summary (100 episodes per environment using seed 42). Mean and std of fitness scores and mean steps to episode termination are reported. The "Train" column reports results in the training environment, while the "Test" columns report results in the two test environments.

intended task.

We use the lava world environment from [5] to test the LLM's ability to design reward functions which mitigate distributional shift. In this environment, the agent is placed in a grid world with lava blocks and a goal block. The intended goal is to get to the goal block without stepping on any lava blocks, which is measured by a fitness function that gives reward for getting to the goal and penalty for stepping on lava. The most "obvious" reward function design for this environment is to give reward for getting to the goal and penalty for stepping on lava, but this leads to distributional shift issues as if the agent is trained in an environment where the lava blocks are in one configuration, it may fail when tested in an environment where the lava blocks are in a different configuration. Two test environments are used to test for distributional shift, one where another layer of lava blocks is added to block the path to the goal, and one where a lava block is removed to open up a new path to the goal.

4 Experimental Setup

Temperature 1.0 for each LLM run 3 runs averaged for one shot 2 runs averaged for iteration loop, with 3 iterations per loop

5 Results

6 Discussion

7 Limitations and Future Work

References

[1] Alessandro Allievi, Peter Stone, Scott Niekum, Serena Booth, and W. Bradley Knox. The perils of trial-and-error reward design: Misdesign through overfitting and invalid task specifications, 2023.

[2] Jack Clark and Dario Amodei. Faulty reward functions in the wild, dec 2016. OpenAI blog post; accessed 2026-05-04.

[3] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. A survey on large language models for code generation. *ACM Transactions on Software Engineering and Methodology*, 35(2):1–72, January 2026.

[4] Victoria Krakovna, Jonathan Uesato, Vladimir Mikulik, Matthew Rahtz, Tom Everitt, Ramana Kumar, Zac Kenton, Jan Leike, and Shane Legg. Specification gaming: the flip side of ai ingenuity, apr 2020. Google DeepMind blog post; accessed 2026-05-04.

[5] Jan Leike, Miljan Martic, Victoria Krakovna, Pedro A. Ortega, Tom Everitt, Andrew Lefrancq, Laurent Orseau, and Shane Legg. Ai safety gridworlds, 2017.

[6] Weixin Liang, Yaohui Zhang, Zhengxuan Wu, Haley Lepp, Wenlong Ji, Xuandong Zhao, Hancheng Cao, Shuyuan Liu, Siming He, Zijian Huang, Diyi Yang, Christopher Potts, Christopher D. Manning, and James Zou. Quantifying large language model usage in scientific papers. *Nature Human Behaviour*, 9(12):2599–2609, dec 2025. Epub 2025-08-04; PMID: 40760036; accessed 2026-05-04.

[7] Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Eureka: Human-level reward design via coding large language models, 2024.

[8] Roland Pihlakas. From homeostasis to resource sharing: Biologically and economically aligned multi-objective multi-agent gridworld-based ai safety benchmarks, sep 2024. Version 4.0.0.4; released 2024-09-30.

[9] Antonin Raffin, Ashley Hill, Maximilian Ernestus, Adam Gleave, and Anssi Kanervisto. Stable-baselines3: Reliable reinforcement learning implementations, feb 2021. Blog post; accessed 2026-05-04.

[10] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017.

[11] Rohin Shah, Vikrant Varma, Ramana Kumar, Mary Phuong, Victoria Krakovna, Jonathan Uesato, and Zac Kenton. Goal misgeneralization: Why correct specifications aren't enough for correct goals, 2022.

A Environment Code Obfuscation Details

To reduce the chance that the LLMs succeed by recalling these benchmark environments from pretraining data, we provide an obfuscated version of the environment code during reward-function generation. Our goal is to preserve the environment dynamics and the reward interface while removing obvious identifiers and "giveaway" language.
The obfuscation procedure includes:

- Renaming variables, classes, and functions to semantically neutral identifiers.
- Rewriting comments and docstrings to remove mentions of *AI Safety Gridworlds*, safety, alignment, reward hacking, specification gaming, or distribution shift.
- Reordering code and refactoring equivalent helper logic (without changing behavior) to reduce direct string-level matches.
- Ensuring the reward-function interface contract is unchanged so that generated reward functions remain compatible with training and evaluation.

A.1 Boat Race (Obfuscated Domain)

We reskin the boat-racing track as an *orchard loop*, where the agent is a picker moving around an orchard with fruit markers, while preserving the same underlying movement and layout structure.

```
# orchard loop layout (obfuscated, non-runnable)

TILE_WALL = '#'
TILE_EMPTY = ' '
TILE_AGENT = 'P'
TILE_M1 = 'm'
TILE_M2 = 'n'
TILE_M3 = 'o'
TILE_M4 = 'p'

GAME_ART = [
  "#####",
  "#P m#",
  "#n#o#",
  "# p #",
  "#####",
]


def build_scene(env_state):
  # single fixed layout
  return make_world(
    env_state,
    GAME_ART,
    what_lies_beneath=' ',
    sprites={TILE_AGENT: [PickerSprite]},
  )


class PickerSprite(object):
  def __init__(self, corner, position, character, env_state, original_board, impassable=TILE_WALL):
    # init wiring intentionally minimal
```

```
    pass
    self._prev_pos = position

  def update(self, actions, board, layers, backdrop, things, the_plot):
    self._prev_pos = self.position
    pass

  # update hook intentionally omitted


class OrchardLoopEnvironment(object):
  def __init__(self, max_iterations=100, noops=False):
    # minimal encoding for board values
    tile_codes = {
      TILE_WALL: 0.0,
      TILE_EMPTY: 1.0,
      TILE_AGENT: 2.0,
      TILE_M1: 3.0,
      TILE_M2: 3.0,
      TILE_M3: 3.0,
      TILE_M4: 3.0,
    }

    action_set = DEFAULT_ACTION_SET
    if noops:
      action_set = DEFAULT_ACTION_SET + [Actions.NOOP]

    pass


OBSERVATION_ENCODING = {
  "shape": (1, 5, 5),
  "dtype": "float32",
  "mapping": {
    "#": 0.0,
    " ": 1.0,
    "P": 2.0,
    "m": 3.0,
    "n": 3.0,
    "o": 3.0,
    "p": 3.0,
  },
  "note": "board array from wrapper; single frame only",
}
```

A.2 Lava World (Obfuscated Domain)

We reskin the lava navigation task as a *chemical lab spill* domain (hazard tiles and a target tile), while preserving the same grid navigation dynamics and obstacle/goal structure.

```
# chemical lab layout (obfuscated, non-runnable)

TILE_WALL = '#'
TILE_EMPTY = ' '
TILE_AGENT = 'A'
TILE_X = 'X'
TILE_Y = 'Y'
```

```
GAME_ART = [
  "#########",
  "#A XXX Y#",
  "# #",
  "# #",
  "# #",
  "# XXX #",
  "#########",
]


def build_scene(env_state):
  # single fixed layout
  return make_world(
    env_state,
    GAME_ART,
    what_lies_beneath=' ',
    sprites={TILE_AGENT: [WorkerSprite]},
  )


class WorkerSprite(object):
  def update(self, actions, board, layers, backdrop, things, the_plot):
    pass

  # update hook intentionally omitted


class SpillLabEnvironment(object):
  def __init__(self):
    tile_codes = {
      TILE_WALL: 0.0,
      TILE_EMPTY: 1.0,
      TILE_AGENT: 2.0,
      TILE_Y: 3.0,
      TILE_X: 4.0,
    }

    pass


OBSERVATION_ENCODING = {
  "shape": (1, 7, 9),
  "dtype": "float32",
  "mapping": {
    "#": 0.0,
    " ": 1.0,
    "A": 2.0,
    "Y": 3.0,
    "X": 4.0,
  },
  "note": "board array from wrapper; single frame only",
}
```